Vulcan

Supplementary slides. AMD AI DevMaster Hackathon, Track 2, Private AI Agents.

A codebase agent that generates every token on hardware you control. One section per slide. Every figure here is reproducible from the committed JSON in `bench-results/` with `vulcan bench-compare`, so nothing in these slides was typed in by hand.

1. The problem

Developers on private or regulated codebases cannot paste source into a cloud LLM. So they either stop using AI assistance, or they leak.

Vulcan is a codebase agent that generates every token on hardware you control, never a third-party LLM API.

2. What it does

Index a repo, then ask it questions in plain language. It searches, reads files, greps, runs tests and answers with `file:line` citations.

Autonomous tool use over a ReAct loop, persistent per-project memory, zero data egress.

3. Architecture

```
user -- CLI -- Agent (ReAct, JSON tool protocol)
                |- RAG index (SQLite + cosine, no vector DB)
                |- Tools (search_code, read_file, grep, run_cmd*, write_file)
                |- Memory (durable per-project notes)
                `- LLM client -- OpenAI-compatible endpoint
                                   `- vLLM on ROCm / Radeon GPU
```

`*` allowlisted commands only, file access is path-jailed.

One environment variable switches the backend. The same agent runs on Ollama during development and vLLM-ROCm in production, which is what made the measurements in slide 5 an apples-to-apples swap.

4. Running on Radeon

vLLM 0.16.1 on ROCm 7.2.1, serving `Qwen/Qwen3-8B` at `--gpu-memory-utilization 0.92`.

The agent needs no code change to target it: `vulcan/llm.py` speaks OpenAI-compatible chat and embeddings against any `base_url`.

One honest caveat: `vllm serve Qwen/Qwen3-8B` runs task=generate and exposes no `/v1/embeddings` route (verified, 404). Radeon Cloud allows one active instance per account, so the measured setup runs generation on the GPU and embeddings on a separate local model. Both are operator-controlled, so no source leaves your machines, but only generation is GPU-served here.

5. Concurrency is the whole argument

A single-stream benchmark hides the thing that matters. An agent fleet issues many ReAct steps at once.

concurrent	laptop (Ollama 4B)	Radeon (vLLM 8B)
1	28.4 chunks/s, TTFT 4.36s	23.5 chunks/s, TTFT 1.49s
2	**20.0** chunks/s, TTFT **42.21s**	**42.9** chunks/s, TTFT **1.25s**
4	not run, curve already inverted	87.0 chunks/s, TTFT 0.55s
8	not run	**179.2** chunks/s, TTFT **0.41s**

- Radeon: **7.63x throughput across an 8x load increase, 95% efficiency.** TTFT *improves* 3.6x. Batch wall clock stays ~57s for 1 request or 8.
- Laptop: one extra request makes aggregate throughput *fall* and TTFT grow nearly 10x. No continuous batching, so requests queue instead of sharing.

At two concurrent requests: **1.2s against 42.2s**, a 34x gap in what the user waits, with the Radeon carrying twice the parameters.

Measured three times across two independent clients: **6.96x, 7.23x, 7.63x**. The 6.96x run is the one visible on screen in the demo video, produced by the shipped CLI rather than by a benchmarking harness written for the occasion.

5b. Two more measured wins

Thinking mode off. Qwen3 reasons by default, which is the wrong trade for an agent: every step pays for a preamble nobody reads.

mode	deltas	wall clock	rate
thinking (default)	4058	170.9s	23.8/s
`enable_thinking: false`	1168	49.0s	24.0/s

Identical rate, 3.5x fewer tokens, so 3.5x less latency per step. A serving-config win, not a faster GPU. Reproducible with `VULCAN_ENABLE_THINKING=false`.

Prefill has a cliff. 512 words costs 0.316s, 2048 costs 0.399s, 8192 costs 2.091s. So the RAG layer has a budget: stay under ~2000 words of retrieved context and TTFT stays under half a second.

6. Honest measurement

- Radeon numbers are measured over an HTTPS proxy (the instance SSH port is not reachable from the client network), so the proxy round-trip is inside every reported TTFT. It was measured separately and is stated.
- "Tokens/sec" counts streamed SSE deltas, not tokenizer tokens.
- Cold start is reported, not hidden: the first two repetitions of each prompt carry cache-warming cost. Median of 5.

7. What is next

- Quantization sweep: fp16 against int8/awq, quality against speed
- vLLM tuning: max-num-seqs, chunked prefill
- Embedding throughput on GPU against CPU